

System Security – Project Report

Fake Data Prevention with Conventional Cryptotools

Project Code: SEC-PRJ-7E_25

Author	Kahathota Liyanage Yasindu Janiya Weerasinghe
Matriculation No.	557466
Degree Programme	Data Analytics
University	University of Messina
Course	System Security

1. Introduction

Data integrity is a fundamental requirement in any system that exchanges information between two or more parties. When a client receives data from a server, it must be able to confirm two things: that the data really came from the expected source, and that the data was not altered along the way. Without these guarantees, an attacker positioned between the two parties could inject fake values or modify legitimate ones without being detected.

This project, **SEC-PRJ-7E_25 – Fake Data Prevention with Conventional Cryptotools**, addresses this problem using established cryptographic primitives rather than custom or experimental techniques. The solution combines RSA digital signatures, X.509 certificates, and JSON Web Tokens (JWT) to build a client-server system in which the server signs every piece of data it produces and the client independently verifies that signature before trusting the data.

The central idea is simple but powerful: if even a single byte of the data is changed after it has been signed, the signature verification fails. This converts the abstract goal of “detecting fake data” into a concrete, testable mechanism.

1.1 Objectives

- Build a working client-server prototype in Python that signs and verifies data.
- Use RSA-2048 digital signatures to guarantee authenticity and integrity.
- Use an X.509 certificate to distribute the server's public key to the client.
- Use JWT (RS256) as the transport format for signed data.
- Demonstrate, through tests, that valid data is accepted and tampered data is rejected.

1.2 Scope

The project focuses on the integrity and authenticity of data in transit between a server and a client on the same machine or local network. Confidentiality (encrypting the payload so it cannot be read) is outside the core scope, since the goal is fake-data prevention rather than secrecy. No specialised hardware is required; the entire system runs on a standard laptop.

2. Background and Cryptographic Concepts

2.1 Digital Signatures

A digital signature is a cryptographic value that binds a message to the identity of its signer. It relies on asymmetric (public-key) cryptography, where each party holds a key pair: a private key that is kept secret, and a public key that can be freely shared. The signer uses the private key to produce a signature over the message; anyone holding the matching public key can verify that signature. Crucially, the signature cannot be produced without the private key, and any change to the message invalidates it.

2.2 RSA-2048

RSA is one of the most widely used asymmetric algorithms. The number 2048 refers to the key length in bits, which is the current industry-standard minimum for secure RSA keys. In this project, signing is performed by hashing the data with SHA-256 and then applying the RSA private key to the resulting hash. Verification reverses the process using the public key.

2.3 X.509 Certificates

A public key on its own is just a number; it does not say who it belongs to. An X.509 certificate solves this by wrapping a public key together with identifying information (such as the organisation and common name) into a structured, signed document. In this project the server generates a self-signed certificate, which is appropriate for a controlled academic environment. The client reads this certificate to obtain a trusted copy of the server's public key.

2.4 JSON Web Tokens (JWT)

A JWT is a compact, URL-safe token format consisting of three parts separated by dots: a header, a payload, and a signature. When signed with the RS256 algorithm, the signature is an RSA signature over the header and payload using SHA-256. JWT is used here as the transport container: the server places the data in the payload and signs the whole token, and the client verifies the token using the server's public key. If the payload is modified after signing, the RS256 signature no longer matches and verification fails.

3. System Architecture

The system follows a straightforward client-server model. The server is the authority that produces and signs data; the client is the consumer that verifies it. All communication happens over HTTP, with the server exposing a small REST API.

3.1 Components

Server side:

- **RSA-2048 private key** – the server's secret, used to sign data. Never leaves the server.
- **X.509 self-signed certificate** – contains the matching public key, served to clients.
- **Digital signer (SHA-256)** – produces the signature over each payload.
- **JWT issuer (RS256)** – packages the signed payload into a JWT.
- **Flask REST API** – exposes endpoints on port 5000.

Client side:

- **Public key extraction** – reads the server's certificate and pulls out the public key.
- **Certificate handling** – loads and parses the X.509 certificate.
- **JWT validator (RS256)** – verifies the token signature against the public key.
- **Data integrity check** – concludes whether the received data is authentic or tampered.

3.2 Data Flow

The end-to-end flow proceeds in the following steps:

1	The server generates an RSA-2048 key pair and a self-signed X.509 certificate (one-time setup).
2	The private key is stored securely on the server; the public key is embedded in the certificate.
3	A client requests the server's certificate and extracts the public key from it.
4	The client requests data; the server signs the payload and returns it as an RS256 JWT.
5	The client verifies the JWT signature using the extracted public key.
6	If the signature is valid the data is accepted; if it was tampered with, verification fails and the data is rejected.

3.3 REST API Endpoints

Endpoint	Method	Purpose
/certificate	GET	Returns the server's X.509 certificate (PEM).
/data	GET	Returns a valid RS256 JWT with a signed payload.
/data/tampered	GET	Returns a JWT whose payload was altered after signing (for testing).

4. Implementation

The system was implemented in Python 3 using three well-established libraries: **cryptography** for key generation, X.509 certificates, and signing; **PyJWT** for creating and verifying JWTs; and **Flask** for the REST API. The client uses the **requests** library for HTTP calls. The codebase is organised into four scripts.

4.1 Project Structure

```
sec_prj_7e/
  keygen.py          RSA-2048 key pair + self-signed X.509 certificate
  server.py          Flask API: signs data, issues JWTs
  client.py          verifies JWT signature using the certificate
  tamper_test.py     standalone test suite (no server required)
  keys/
    private_key.pem
    public_key.pem
  certs/
    server_cert.pem
```

4.2 Key and Certificate Generation (keygen.py)

This script runs once before the system is used. It generates a 2048-bit RSA key pair, saves the private and public keys as PEM files, and builds a self-signed X.509 certificate valid for 365 days. The certificate identifies the project organisation and is signed with SHA-256.

```
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048,
)
```

4.3 Signing and Issuing Data (server.py)

The server loads its private key at startup. When a client calls `/data`, the server builds a sample sensor payload and signs it into a JWT using the RS256 algorithm. The signature covers the entire header and payload, so the token is self-protecting.

```
token = jwt.encode(
    payload,
    PRIVATE_KEY_PEM,
    algorithm="RS256",
)
```

The `/data/tampered` endpoint deliberately demonstrates an attack. It signs a legitimate payload, then modifies a value inside the token while keeping the original signature. Because the signature no longer matches the altered payload, any correct verifier will reject it.

4.4 Verifying Data (client.py)

The client first downloads the server's certificate and extracts the public key from it. It then requests data and verifies the JWT. A successful verification returns the decoded payload; a failed one raises an `InvalidSignatureError`, which the client reports as tampered data.

```
cert = x509.load_pem_x509_certificate(resp.content)
public_key = cert.public_key()
...
decoded = jwt.decode(token, public_pem, algorithms=["RS256"])
```

5. Testing and Results

Two forms of testing were carried out: a standalone test suite that runs without a server, and a live end-to-end test between the running server and the client.

5.1 Standalone Test Suite (`tamper_test.py`)

The test suite exercises five scenarios covering both the success case and several distinct tampering attempts. Each test prints a clear PASS or FAIL result.

```
[PASS] valid data accepted
[PASS] tampered value rejected
[PASS] tampered sensor_id rejected
[PASS] wrong key rejected
[PASS] expired token rejected
```

Figure 1. Output of the standalone test suite – all five scenarios pass.

The five scenarios verify the following properties:

Scenario	Expected	Property Tested
Valid data	Accepted	Correct data verifies successfully.
Tampered value	Rejected	A changed numeric field breaks the signature.
Tampered sensor_id	Rejected	A changed identifier breaks the signature.
Wrong key	Rejected	A signature from a different key is not trusted.
Expired token	Rejected	Tokens past their expiry are not accepted.

5.2 Live End-to-End Test (server + client)

With the server running, the client was executed against the live API. The client successfully extracted the public key from the certificate, accepted the valid token, and rejected the tampered one.

```
Public key extracted from server certificate
-----
/data          -> VALID
  payload: {'sensor_id': 'TEMP-SENS-014',
            'location': 'Messina-Lab-A',
            'value': 23.7, 'unit': 'C', ...}
-----
/data/tampered -> TAMPERED / FAILED:
  Signature verification failed
```

Figure 2. Live client output against the running server.

These results confirm the core objective of the project. Legitimate, unaltered data passes verification and is accepted. Data that has been modified after signing – even by a single field – fails verification and is rejected, exactly as required for fake-data prevention.

6. Discussion

6.1 Security Properties Achieved

- **Integrity:** any modification to the data invalidates the signature.
- **Authenticity:** only the holder of the private key can produce valid signatures.

- **Verifiability:** any client with the certificate can independently check data without secrets.

6.2 Limitations

The certificate is self-signed, which is acceptable for a closed academic setting but would not be trusted automatically in a public deployment, where a recognised Certificate Authority would be used instead. The system also focuses on integrity and authenticity rather than confidentiality; the payload itself is readable. Finally, the prototype runs on Flask's development server, which is not intended for production traffic.

6.3 Possible Extensions

- Encrypt the payload as well, to add confidentiality alongside integrity.
- Use a proper Certificate Authority and certificate chain validation.
- Add key rotation and certificate revocation handling.
- Deploy behind a production-grade WSGI server with HTTPS.

7. Conclusion

This project successfully demonstrates how conventional cryptographic tools can be combined to prevent fake or tampered data in a client-server system. By using RSA-2048 digital signatures, an X.509 certificate for public-key distribution, and JWT as a signed transport format, the system guarantees that any tampering with the data is reliably detected. The testing confirmed that valid data is accepted while every tampering scenario is rejected. The approach relies entirely on well-understood, standardised primitives, making it both robust and straightforward to reason about.

8. References

- 1 R. L. Rivest, A. Shamir, and L. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," Communications of the ACM, 1978.
- 2 Internet Engineering Task Force (IETF), RFC 5280: Internet X.509 Public Key Infrastructure Certificate and CRL Profile.
- 3 Internet Engineering Task Force (IETF), RFC 7519: JSON Web Token (JWT).
- 4 Python Cryptographic Authority, "cryptography" library documentation, <https://cryptography.io>.
- 5 Jöse / PyJWT documentation, <https://pyjwt.readthedocs.io>.
- 6 Pallets Project, Flask documentation, <https://flask.palletsprojects.com>.